Environmental Modelling and Software 199 (2026) 106909

Contents lists available at ScienceDirect

Environmental Modelling and Software

journal homepage: www.elsevier.com/locate/envsoft

Building and managing local databases from Google Earth Engine with the `geeLite` R package

Marcell T. Kurbucz [a,b,*,1], Bo Pieter Johannes Andrée [b,1]

[a] Institute for Global Prosperity, The Bartlett, University College London, 9–11 Endsleigh Gardens, London, WC1H 0EH, United Kingdom
[b] Development Economics Data Group, World Bank, 1818 H Street NW, Washington, D.C., 20433, USA

ARTICLE INFO

JEL classification:
C21
C63
C81
C88
L17

Keywords:
Google Earth Engine
Geographic information system
Remote sensing
Raster data
Spatio-temporal data
Software

ABSTRACT

Google Earth Engine (GEE) has substantially expanded the scope of geospatial analysis by providing access to petabyte-scale satellite imagery and geospatial data together with cloud-based computation. This accessibility enables researchers to study large-scale environmental and socio-economic processes with high spatial and temporal coverage. In recent years, numerous R packages have emerged to leverage GEE's functionalities. However, constructing and managing complex spatio-temporal databases for continuous monitoring of remotely sensed data remains challenging and often requires advanced coding skills. To bridge this gap, we introduce the `geeLite` R package (available on CRAN), which supports the construction and updating of local databases for GEE-derived outputs, enabling users to track their evolution over time. By storing results in SQLite format — a serverless, self-contained database solution requiring no additional setup — `geeLite` simplifies data collection. It also streamlines conversion to native R formats and provides functions to aggregate and process the resulting databases for downstream analysis.

1. Introduction

The ever-growing volume of Earth observation data presents both opportunities and challenges for scientific inquiry. While vast datasets hold the potential to revolutionize our understanding of Earth systems, traditional desktop-based analysis methods often struggle with the computational burden associated with such data (Amani et al., 2020). Google Earth Engine (GEE) has emerged as a powerful solution, offering a cloud-based platform for efficient management, analysis, and visualization of geospatial big data (Gorelick et al., 2017). Its core strength lies in its ability to overcome the limitations of traditional approaches by providing the following features (Tamiminia et al., 2020):

- **A comprehensive geospatial data catalog:** Petabytes of public geospatial data readily accessible through a web interface, including historical and current satellite imagery, environmental variables, and other geospatial information.
- **High-performance parallel processing:** The ability to leverage Google's cloud infrastructure for large-scale analysis, enabling researchers to tackle complex problems that would be infeasible on personal computers.
- **An accessible development environment:** A web-based interface and application programming interfaces (APIs) in JavaScript and Python, supporting widely-used programming languages.

This combination of features allows scientists to investigate new scientific questions in a wide range of subjects, particularly those requiring large-scale spatial and temporal analysis.[2] Examples include studies on climate change (Banerjee et al., 2024; Kazemi Garajeh et al., 2024), environmental

* Corresponding author at: Institute for Global Prosperity, The Bartlett, University College London, 9–11 Endsleigh Gardens, London, WC1H 0EH, United Kingdom.
E-mail addresses: m.kurbucz@ucl.ac.uk (M.T. Kurbucz), bandree@worldbank.org (B.P.J. Andrée).

[1] These authors contributed equally to this work. Funding by the World Bank's Food Systems 2030 (FS2030) Multi-Donor Trust Fund program (TF0C0728 and TF0C7822) is gratefully acknowledged. We thank Andres Chamorro and Ben P. Stewart for code testing and comments, as well as Steve Penson, David Newhouse, and Alia J. Aghjanian for helpful comments and input. This paper reflects the views of the authors and does not reflect the official views of the World Bank, its Executive Directors, or the countries they represent.

[2] For a global bibliometric overview of GEE research trends and future directions, see Velastegui-Montoya et al. (2023); for a broad thematic review of GEE capabilities and applications, with a focus on developing-country contexts, see Vijayakumar et al. (2024).

https://doi.org/10.1016/j.envsoft.2026.106909
Received 18 May 2025; Received in revised form 29 January 2026; Accepted 9 February 2026
Available online 23 February 2026

Table 1
Metadata of the `geeLite` package.

Metadata description	Metadata contents
Permanent link (CRAN):	https://CRAN.R-project.org/package=geeLite
DOI:	https://doi.org/10.32614/CRAN.package.geeLite
Source code repository:	https://github.com/mtkurbucz/geeLite
Legal code license:	Mozilla Public License Version 2.0 (MPL-2.0)
Software code languages:	R
Code versioning system:	Git
Current code version:	1.0.6
System requirements:	OS agnostic (Linux, macOS, MS Windows). R 4.3.2 or later.
R packages (Imports):	`cli`, `crayon`, `data.table`, `dplyr`, `geojsonio`, `googledrive`, `h3jsr`, `jsonlite`, `knitr`, `lubridate`, `magrittr`, `progress`, `purrr`, `reshape2`, `reticulate`, `rgee`, `RSQLite`, `sf`, `stats`, `stringr`, `tidyr`, `utils`, `rnaturalearth`, `rnaturalearthdata`, `rstudioapi`.
R packages (Suggests):	`leaflet`, `optparse`, `rmarkdown`, `testthat` ($\geq$ 3.0.0), `withr`.
External dependencies:	A virtual Conda environment with the following Python packages: `earthengine-api`, `ee_extra`, and `numpy`. The current version of `geeLite` (1.0.6) specifically requires `earthengine-api` version 0.1.370.
Support email:	m.kurbucz@ucl.ac.uk and bandree@worldbank.org

Note: Packages listed under Suggests are optional and used only for vignettes, examples, and development-time testing; they are not required for `geeLite`'s core workflow. The `earthengine-api` version is held fixed to ensure reproducibility and compatibility with `rgee`; it may be updated in future `geeLite` releases once compatibility with newer versions has been verified.

degradation (Andrée et al., 2019), land cover change (Wang et al., 2020; Burgueño et al., 2023), deforestation (Chen et al., 2021; Brovelli et al., 2020), flood monitoring (Hamidi et al., 2023), wildfire prediction (Tavakkoli Piralilou et al., 2022), urbanization (Marconcini et al., 2021; Zheng et al., 2021), food security (Andrée et al., 2020; Wang et al., 2022; Penson et al., 2024), and sustainable development (Burke et al., 2021), to name a few.

Recognizing the broader potential of GEE within the R user community, Aybar et al. (2020) developed the `rgee` R package. Acting as a bridge, `rgee` seamlessly integrates GEE with R's extensive ecosystem of geospatial packages, making GEE's capabilities accessible to a wider user base. This integration is facilitated by the `reticulate` package (Ushey et al., 2024), allowing `rgee` to interact with GEE's official Python API and utilize all its modules, classes, and functions.

In recent years, numerous R packages have been published to extend the functionality of `rgee` and improve its user experience. Among these, `tidyrgee` (Arno and Erickson, 2022) aids users in filtering, joining, and summarizing GEE image collections, `rgee2` (Kong, 2022) expands upon the original package with additional utility functions, and `rgeeExtra` (Aybar et al., 2023) provides more R-like syntax wrappers and additional extension methods for common Earth Engine classes. Other packages, like `SAEplus` (SSA Statistical Team, 2022) and `LandsatTS` (Berner et al., 2023), facilitate the data extraction and processing from GEE or provide geospatial decision-making tools, such as `RePlant alpha` (Morales et al., 2023).

While available third-party libraries improve the accessibility of GEE's functionalities, constructing and managing up-to-date spatio-temporal databases for continuous monitoring applications remains a complex, time-consuming task that requires advanced coding knowledge. In current practice, these tasks are typically handled through ad hoc scripts written directly with the GEE API or with `rgee`. Users must manually manage batch processing to respect API limits, export results (often through Google Drive), merge partial files into a consistent local schema, and keep track of which regions and time periods have already been processed. Such manual workflows are feasible but difficult to reproduce, and even small inconsistencies in naming or export handling can lead to fragmented or outdated databases. This poses a significant barrier for practitioners tasked with environmental monitoring but lacking the time or technical expertise to maintain complex infrastructure.

To address this gap, our paper introduces `geeLite`, a novel R package that automates these operational steps and supports reproducible local database management for GEE-derived geospatial indicators. Through a declarative configuration file, `geeLite` enables users to build, maintain, and update local spatio-temporal databases of GEE-computed features, while automatically handling batching, exports, and state management. Results are stored in SQLite format—a serverless, self-contained solution that eliminates the need for additional setup or administration.[3] Furthermore, the package streamlines conversion to native R formats and provides utilities for aggregating and processing the resulting databases for downstream analysis. The metadata for the package is detailed in Table 1.

The rest of this paper is organized as follows. Section 2 situates `geeLite` within the broader GEE and geospatial software ecosystem. Section 3 describes the package's installation, workflow, structure, and testing. Section 4 presents an illustrative example, and Section 5 discusses recommended use, scalability, and limitations. Section 6 concludes and outlines potential directions for future development.

2. Context and related tools

Beyond the R ecosystem, several platforms offer user-friendly interfaces to Google Earth Engine. These tools support a wide range of interactive analysis, visualization, and preprocessing tasks, and excel at streamlining exploratory workflows. However, they are typically not designed for the kind of configuration-driven local database management provided by `geeLite`. In Python, `geemap` (Wu et al., 2019; Wu, 2020) and `geetools` (GEE Community, 2025b) facilitate interactive or batch operations on GEE data but focus on visualization and preprocessing rather than reproducible local databases. Unlike `geemap` or `geetools`, `geeLite` is database-first: it is designed to produce and maintain a persistent, updatable local SQLite database with aggregated and derived statistics as the main output, rather than transient in-session objects or one-off export files. In desktop GIS, recent integrations — such as the Google Earth Engine Plugin for QGIS (GEE Community, 2025c) and the ArcGIS Earth Engine

[3] More information can be found at: https://sqlite.org/features.html (retrieved: December 1, 2025).

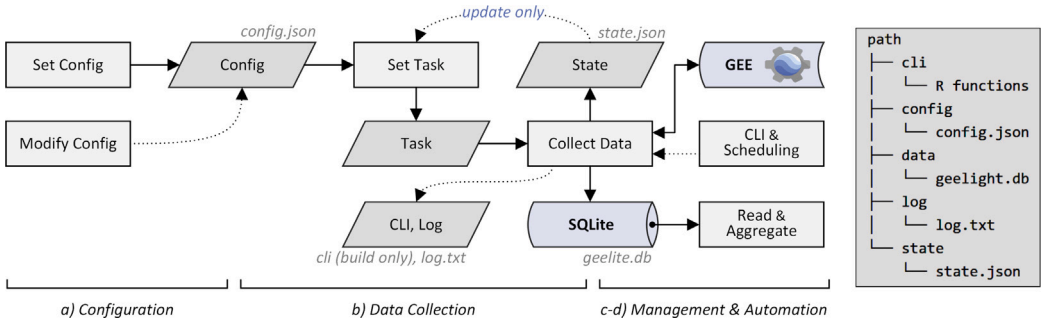

Fig. 1. Workflow of `geeLite` and the folder structure of the generated database.
Note: The workflow consists of (a) configuration, where users specify the database settings, and (b) authenticated extraction from GEE to create or refresh the local SQLite database. Steps (c) and (d) provide utilities for working with the data in R and for automating recurring runs via the command-line interface (CLI).

Toolbox (GEE Community, 2025a) — provide GUI-oriented (low-code) access to GEE within familiar graphical workflows. These tools improve accessibility for non-programmers but remain oriented toward interactive, project-based analysis. By contrast, `geeLite` emphasizes automation; reproducible, verifiable workflows for data aggregation and indicator construction; and lightweight storage via a scriptable R workflow that can be scheduled and version-controlled.

Commercial remote-sensing suites have taken similar steps: ERDAS IMAGINE's LiveLink for GEE (Hexagon Geospatial, 2024) integrates cloud processing into its visual Spatial Modeler environment. To our knowledge, ENVI does not currently provide an official Google Earth Engine connector; workflows using GEE outputs in ENVI therefore typically rely on manual exports or custom scripting. Compared to these systems, `geeLite` provides an open-source, vendor- and GIS-platform-agnostic alternative that combines the efficiency of cloud processing with the transparency of local database management. Its emphasis on configuration-driven automation and portable SQLite outputs distinguishes it from GUI-driven approaches, addressing a practical gap between ad hoc scripting and enterprise GIS infrastructure.

Conceptually, `geeLite` represents a distinct architectural choice within the geospatial data ecosystem. Many GEE-based pipelines run entirely inside a single cloud platform, while large-scale local workflows often rely on distributed frameworks such as Dask (Rocklin et al., 2015), Apache Beam (Apache Beam Community, 2025), or Spark (Zaharia et al., 2016). In contrast, `geeLite` follows a hybrid model: it delegates heavy computation to the GEE cloud, but handles aggregation, state management, and reproducible storage locally through a lightweight SQLite-based layer. In this sense, the approach offloads computation to the cloud while retaining a version-controlled local trace of results—analogous in spirit to query pushdown in systems such as Trino (Fuller et al., 2022). This is particularly valuable in high-stakes monitoring (e.g., early-warning, parametric insurance triggers, or humanitarian decision support), where users need to audit how an indicator was produced rather than only consuming final cloud outputs. By combining these roles, `geeLite` fills a practical gap between monolithic cloud workflows and distributed local computation frameworks, offering a lightweight, reproducible solution for recurring indicator-based analyses.

3. Software description

3.1. Installation

The `geeLite` package is available on CRAN and can be installed using the standard R installer. A development version is maintained on GitHub (Kurbucz and Andrée, 2024) and can be installed via `devtools` (Wickham et al., 2022). To interact with GEE, `geeLite` builds on `rgee`, which relies on a dedicated Conda environment containing the `earthengine-api`, `ee_extra`, and `numpy` Python packages. After installing `geeLite`, users can set up this environment interactively using the `gee_install` function, which confirms environment changes with the user and registers the resulting Python setup for `rgee`. In some Conda installations, users may also need to manually accept Conda's Terms of Service before package installation can proceed; `gee_install` provides guidance if this step is required.

```
# Install geeLite package:                                    [R code]
install.packages("geeLite")
geeLite::gee_install()
```

Note that the current version of the `geeLite` package (1.0.6) is specifically compatible with `earthengine-api` version 0.1.370. The `gee_install` function installs this pinned version alongside `ee_extra` and `numpy`, and registers the resulting Python setup for `rgee`.

3.2. Workflow

The `geeLite` workflow has two core stages. In the configuration stage (a), users define the spatial units, variables, and aggregation settings in a JSON file. This file then drives the data-collection stage (b), where `geeLite` authenticates with GEE and executes the extraction to either create the local SQLite database or extend it with newly available observations. The package also provides tools for working with the resulting data in R (c) and offers utilities to facilitate automated database updates (d). Fig. 1 summarizes the workflow and the folder structure of the generated database.

(a) *Configuration*

A configuration file, created using the `set_config` function, is stored locally in JSON format. This file allows users to define regions of interest (`regions`) for zonal statistics calculations at both the administrative level 0 (countries) and administrative level 1 (subnational states). These regions are identified using ISO standards: countries are specified by ISO 3166-1 alpha-2 codes (two letters), and first-level subdivisions

by ISO 3166-2 codes (country code plus additional characters).[4] Users can select multiple data sources (`source`) by specifying one or more GEE datasets, along with their bands of interest and the statistical indicators for spatial aggregation.

The configuration file also includes settings for the resolution of the H3 grid system (`resol`),[5] the start date for data collection (`start`), and a scaling parameter to adjust image quality before processing (`scale`). Additionally, to provide a safeguard against generating overly large exports — which can lead to GEE task failures — users can set a maximum total number of features (e.g., H3 grid cells) to be processed in the configuration file using the `limit` parameter. The default limit is set to 10,000 features.

To simplify the configuration process for data collection, the `get_state` function allows users to view the complete list of available region codes, while the `get_config` function displays the current contents of the configuration file. Users can also modify the selected regions (`regions`), data sources (`source`), and the limit on concurrent zonal statistics calculations (`limit`) either manually (outside the R session) or using the `modify_config` function. However, modifying other configuration parameters requires rebuilding the database based on a new configuration file.

(b) *Data Collection (with Authentication)*

Data collection is performed with `run_geelite`, which handles user authentication and then builds a new database or updates an existing one.

Authentication

Building on the authentication protocol of the `rgee` package, `run_geelite` initializes the Earth Engine Python API via `set_depend`. On startup, `geeLite` first attempts to initialize Earth Engine using any existing cached Earth Engine credentials. If this initialization fails (e.g., no valid token is found), `run_geelite` falls back to the interactive browser-based authentication workflow provided by `rgee`, where users grant permission to link their Google accounts with GEE and paste the resulting token into R.

Credentials are stored following `rgee`/Earth Engine conventions under `~/.config/earthengine/`, optionally in a user-named subdirectory when a username is supplied. As long as these credentials remain valid, `geeLite` will automatically reuse them to initialize the selected Google account in subsequent sessions. Note that access to GEE requires a Google Cloud project that is registered for Earth Engine. Accordingly, users should authenticate with the Google account that owns (or has been granted access to) the intended Cloud project, and make sure that the project is enabled for Earth Engine before running `geeLite` workflows.

In addition to the standard authentication method, users may authenticate using Google Cloud service account credentials. This method allows them to create a service account via the Google Cloud Console and download a corresponding JSON key file. By configuring the `GOOGLE_APPLICATION_CREDENTIALS` environment variable to reference the downloaded JSON key file, users can bypass the interactive, browser-based authentication process.[6] `geeLite` does not manage service-account keys directly; instead, it relies on the Earth Engine Python API to detect this environment variable and authenticate non-interactively. This method is particularly useful for automated workflows and production environments where graphical interfaces may not be available.

Database Building and Updating

After authentication, `geeLite` gathers GEE-computed geospatial features based on the configuration file and stores them in SQLite format (*geelite.db*).[7] Depending on the value of the `mode` parameter, the extraction process operates in either `"local"` mode — processing data in smaller chunks directly within R — or `"drive"` mode, which leverages Google Drive for exporting larger datasets. The resulting database contains information about H3 bins (referred to as `grid`) and collected datasets, organized into separate tables named after the corresponding GEE dataset IDs. The `grid` table includes an Open Geospatial Consortium (OGC) feature identifier (`ogc_fid`), an H3 index (`id`), a corresponding region code (`iso`), and a geometry shape (`geometry`). Other tables, containing zonal statistics, are structured in a wide format. These tables include the H3 index (`id`) of the associated bins, the band name (`band`), the statistical indicator employed (`zonal_stat`), and the zonal statistics computed on specific dates (formatted as `YYYY_MM_DD`) when the dataset was updated.

In addition to the configuration file and SQLite database, the package generates two supplementary files: *state.json* and *log.txt*. These files respectively record the current state of the database and log session types (build or update) along with their timestamps. To facilitate dataset management and scheduled updates, the `geeLite` package organizes a `cli` folder alongside the database. This folder contains an R script for each main function of the package — except for `set_cli` and `read_db` — allowing them to be executed from the CLI. To run these functions via the CLI, use the following structure: `Rscript [cli/function] --parameter [parameter]`. These scripts automatically use the path to the root directory of the generated database as an input parameter, eliminating the need for manual definition.[8]

Once the database is built, it can be updated (`rebuild = FALSE`) or rebuilt (`rebuild = TRUE`) by re-running `run_geelite`. Here, `rebuild` is an argument of `run_geelite`: leaving it at its default `FALSE` performs an incremental update using the existing *state.json*, whereas setting it to `TRUE` forces a full rebuild from the current configuration. For updates, the process begins with a comparison of the configuration and state files. If the configuration file has been modified since the last data collection procedure, the database will be adjusted and updated accordingly. A successful update overwrites the state file and modifies the log file. In the case of rebuilding, the data collection is based solely on the configuration file, and it overwrites the existing database and its supplementary files.

> Note that while some parameters of the configuration file (`regions`, `source`, and `limit`) can be modified manually, the **state file is vulnerable to any manual changes**. Manual modification or deletion of this file can corrupt future updates of the database and necessitate a rebuild.

[4] Shapefiles for the selected regions are retrieved using the `rnaturalearth` package (Massicotte and South, 2024).

[5] `geeLite` applies Uber's H3 grid system (Uber, 2021) using the `h3jsr` package (O'Brien, 2023) to divide user-defined regions into hexagonal bins, from which zonal statistics are calculated.

[6] To set this environment variable, use the following R code: `Sys.setenv(GOOGLE_APPLICATION_CREDENTIALS = "path/to/service-account-key.json")`.

[7] SQLite objects are managed by the `RSQLite` package (Müller et al., 2024).

[8] For more detailed information on using these CLI scripts, please refer to Section 4.1 or visit the package's GitHub repository (Kurbucz and Andrée, 2024).

(c) *Data management in R*

The `geeLite` package provides two main functions to simplify the analysis of the generated SQLite database: `fetch_vars` and `read_db`. The `fetch_vars` function allows users to retrieve detailed information about the available variables in the database. This information includes variable names — comprising the database, band, and statistical indicator names separated by slashes — along with variable IDs, source details, start and end dates, and average frequencies in days. The `read_db` function is designed to read, aggregate, and process data from the database according to user-defined parameters.

When executing `read_db` with a non-`NULL` `freq` parameter, the selected variables are first converted to a daily frequency within each spatial unit (`id`). This daily series can be preprocessed using `prep_fun`; by default `prep_fun` performs linear interpolation, but users may supply alternatives (e.g., last-observation-carried-forward imputation, state-space smoothing, or nonparametric models). The preprocessed daily data are then aggregated to the requested frequency via `aggr_funs`. By default, this yields arithmetic means (e.g., monthly means when `freq = "month"`), but users can provide custom functions, including variable-specific or multiple aggregation functions. Finally, post-processing functions (`postp_funs`) are applied after aggregation; if multiple post-processing functions are provided for a variable, each is applied separately to the aggregated time series and returns a distinct output series. When `freq = NULL`, no daily expansion, aggregation, or post-processing is performed and native observation dates are returned.

Customization of aggregation and post-processing functions is optional for standard applications. For advanced projects, however, `geeLite` also supports external configuration and independent versioning of post-processing pipelines. The additional main function `init_postp` allows users to define post-processing functions externally, in separate files, so that more complex logic (including model-based functions) can be developed and versioned independently of the database. Updating the `geeLite` database then simply requires updating these externally defined functions. Concretely, `init_postp` creates a *postp* folder in the root directory of the generated database, containing an editable R script (*functions.R*) where users can define custom post-processing functions, and a JSON file (*structure.json*) that specifies which functions to apply to which variables during post-processing. Further details on the file structure and a worked example are provided in Appendix A.

A useful way to see when this setup helps is to consider maintaining continuous updates of a model-based rainfall-forecast indicator in the central `geeLite` database. The forecasting model can be developed and maintained in a separate R project, allowing easy switching between ARIMA and exponential smoothing (ETS) specifications. The central `geeLite` database then simply calls the relevant externally defined post-processing function and refreshes the rainfall-forecast series using the updated model. This separation of database maintenance from model development is particularly helpful for multidisciplinary data teams and collaborative projects.

After the database is imported, R's full range of capabilities for processing, modeling, and plotting the data can be utilized.

(d) *Automation*

For automation, `geeLite` provides CLI scripts for its main functions. Each generated database includes a *cli* folder with ready-to-use scripts (excluding `set_cli` and `read_db`); the same scripts can also be created separately using `set_cli`. This enables scheduled database updates through job-scheduling utilities such as Linux crontab (see, e.g., Bradley, 2016).

3.3. Structure

The eleven main functions of the `geeLite` package, along with their parameters, are detailed in Table 2.

3.4. Unit tests

The `geeLite` package includes unit tests implemented with the `testthat` package (Wickham, 2011). These tests cover almost all functions in `geeLite`, except `set_cli`, `fetch_regions`, and `init_postp`. They span tasks such as configuration generation, database building and reading, configuration modification, and database updates.

If Google Earth Engine credentials have not been initialized in an interactive `geeLite` session before running the tests, authentication can be triggered manually by calling the internal function `geeLite:::set_depend(conda = "rgee", user = NULL, drive = FALSE, verbose = TRUE)`.[9]

4. Illustrative example

This section presents a practical example of the typical workflow for the `geeLite` package, demonstrating key steps such as configuration, database creation, updates, data retrieval, and processing. The example generates an up-to-date database of NDVI zonal statistics (mean and standard deviation) for Somalia and the Republic of Yemen. To illustrate incremental configuration updates, we first build the database with mean and minimum statistics, and then replace the minimum with standard deviation using `modify_config`. These statistics are calculated from January 1, 2010, using a grid system with a resolution level of three, which corresponds to areas of approximately 12,393 square kilometers.[10]

4.1. Related workflow

(a) *Configuration*

First, the `fetch_regions` function is used to identify the ISO 3166-1 alpha-2 country codes for Somalia and Yemen. These codes are then used to configure the data generation process as follows:

[9] The parameters are described in detail in Table 2.

[10] NDVI data are sourced from the Moderate Resolution Imaging Spectroradiometer (MODIS) instrument. The dataset's profile is available at: https://developers.google.com/earth-engine/datasets/catalog/MODIS_061_MOD13A2 (retrieved: December 1, 2025).

Table 2
Main `geeLite` functions (upper panel) and consolidated definitions of shared parameters (lower panel).

Functions

Name	Parameters	File	Description
`gee_install`	1–3	*gee_install.R*	Creates a Conda environment and installs required dependencies.
`set_config`	4–11	*set_config.R*	Creates the configuration file.
`run_geelite`	1, 4, 11–14	*run_geelite.R*	Collects and stores data, and updates the state and log files.
`modify_config`	4, 11, 15–16	*modify_config.R*	Modifies the configuration file.
`set_cli`	4, 11	*set_cli.R*	Creates scripts to make main functions callable through the CLI.
`get_config`	4	*get_json.R*	Prints the configuration file.
`get_state`	4	*get_json.R*	Prints the state file.
`fetch_regions`	17	*fetch_regions.R*	Displays ISO 3166-1 country and ISO 3166-2 subdivision codes.
`fetch_vars`	2, 18	*access_db.R*	Displays information on the available variables in the SQLite database.
`read_db`	2, 19–23	*access_db.R*	Reads, aggregates, and processes data from the SQLite database.
`init_postp`	4, 11	*access_db.R*	Initializes the file structure for external post-processing.

Parameters

ID	Name	Type	Description
1	`conda`	*character*	Name of the virtual Conda environment.
2	`python_version`	*character*	Python version for the Conda environment (default: `"3.11"`).
3	`force_recreate`	*logical*	If `TRUE`, deletes and recreates the Conda environment even if it already exists (default: `FALSE`).
4	`path`	*character*	Path to the root directory for the generated database.
5	`regions`	*character*	ISO 3166-1 alpha-2 country codes and/or ISO 3166-2 subdivision codes of regions of interest.
6	`source`	*list*	Detailed description of the GEE datasets of interest. This is a nested list with three levels: `names`, `bands`, and `zonal_stats`.[a]
6.1	`names`	*list*	Names of the GEE datasets to be used (e.g., `"MODIS/061/MOD13A2"`).
6.1.1	`bands`	*list*	Specific data bands from the datasets (e.g., `"NDVI"`).
6.1.1.1	`zonal_stats`	*character*	Type of spatial statistics to be computed for each region (options: `"mean"`, `"sum"`, `"median"`, `"min"`, `"max"`, `"sd"`).
7	`resol`	*integer*	Spatial resolution of the H3 grid system.[b]
8	`scale`	*integer*	The nominal resolution (in meters) for processing the image projection. If left as `NULL` (the default), a resolution of 1000 is used.[c]
9	`start`	*date*	Starting date for data collection (default: `"2020-01-01"`).
10	`limit`	*integer*	Controls batch/export size to avoid oversized GEE requests. In `local` mode, the number of features processed per batch is `⌊limit/number of dates⌋`. In `drive` mode, it sets the maximum number of H3 bins per export. Default: 10000.
11	`verbose`	*logical*	Displays messages (default: `TRUE`).
12	`rebuild`	*logical*	If `TRUE`, overwrites the database and associated files based on the configuration file (default: `FALSE`).
13	`user`	*character*	Generates a folder in `~/.config/earthengine/` to store credentials for a specific Google identity. Default is the root directory: `NULL`.
14	`mode`	*character*	Specifies the mode of data extraction. Acceptable values are `"local"` and `"drive"`. In `"local"` mode, data is extracted in smaller chunks directly within R, which is suitable for modest data volumes. In `"drive"` mode, data is exported via Google Drive, enabling the handling of larger datasets that require parallel processing or higher export limits. Defaults to `"local"`.
15	`keys`	*list*	Paths to the values designated for replacement.[d]
16	`new_values`	*list*	New values to replace the original entries at the specified paths.[d]
17	`admin_lvl`	*integer*	Specifies the administrative level: `0` for country, `1` for state, or `NULL` for all regions (default: `0`).
18	`format`	*character*	Specifies the output format. Options include `"data.frame"` (default), `"markdown"`, `"latex"`, `"html"`, `"pipe"` (Pandoc pipe tables), `"simple"` (Pandoc simple tables), or `"rst"`.
19	`variables`	*character or integer*	Names or IDs of the selected variables. Use the `fetch_vars` function to list available variables (default: `"all"`).
20	`freq`	*character*	Output frequency. Options are `"day"`, `"week"`, `"month"` (default), `"bimonth"`, `"quarter"`, `"season"`, `"halfyear"`, or `"year"`; use `NULL` to disable aggregation.
21	`prep_fun`	*function*	Pre-processing applied after conversion to daily frequency and before aggregation; `NULL` (default) triggers linear interpolation.
22	`aggr_funs`	*function or list*	A function or list of functions to aggregate data at the specified frequency (`freq`). The default is `mean`: `function(x) mean(x, na.rm = TRUE)`.[e]
23	`postp_funs`	*function, list, or character*	A function or list of functions applied to the time series data of a single bin after aggregation. The default is `NULL`, indicating no post-processing.[e] Alternatively, set to `"external"` to apply post-processing from external files initialized with the `init_postp` function. See Appendix A for more details.

Note: Parameters marked with underlines are considered optional. (Referenced pages retrieved on December 1, 2025.)

[a] The complete data catalog of GEE is accessible at: https://developers.google.com/earth-engine/datasets/catalog.

[b] Allowable values and related dimensions can be found at: https://h3geo.org/docs/core-library/restable/.

[c] More information can be found at: https://developers.google.com/earth-engine/guides/scale.

[d] For example, use `keys = list("regions", c("source", "MODIS/061/MOD13A2", "NDVI"))` and `new_values = list(c("SO", "KE"), c("mean", "max"))` to update the `regions` to Somalia and Kenya and modify the `zonal_stats` for `"NDVI"` to `"mean"` and `"max"`.

[e] Both `aggr_funs` and `postp_funs` accept a single function or an (optionally nested) list to define variable-specific behavior. Unnamed lists are treated as `"default"` and applied to all variables. Names must exactly match the variable strings returned by `fetch_vars()` (e.g., `"MODIS/061/MOD13A2/NDVI/mean"`) or be `"default"`. For example, `aggr_funs <- list("default" = FUN_1, "Var_1" = list(FUN_1, FUN_2))` applies `FUN_1` to all variables, but overrides `"Var_1"` with `FUN_1` then `FUN_2`. Similarly, `postp_funs <- list("default" = NULL, "Var_1" = FUN_3)` applies `FUN_3` only to that variable after aggregation.

```r
# Setting the config file:
set_config(path = "path/to/db",
           regions = c("SO", "YE"),
           source = list(
              "MODIS/061/MOD13A2" = list(
                 "NDVI" = c("mean", "min")
              )
           ),
           resol = 3,
           start = "2010-01-01")
```
[R code]

As a result, the configuration file (*config/config.json*) is generated at the specified target path.

Alternatively, the data generation process can be configured via the CLI. First, the CLI scripts of the `geeLite` functions are set up using the `set_cli` function. Then, the data generation process is configured through the CLI as follows[11]:

```bash
# Setting the CLI files:
Rscript ./geeLite/cli/set_cli.R --path "path/to/db"

# Change directory:
cd path/to/db

# Setting the configuration file:
Rscript cli/set_config.R --regions "SO YE" --source "list('MODIS/061/
     MOD13A2' = list('NDVI' = c('mean', 'min')))" --resol 3 --start "
     2010-01-01"
```
[Bash code]

(b) *Data Collection*

Once the configuration file is defined, the current database and its supplementary files (*state/state.json*, *log/log.txt*, along with the CLI scripts of the functions) can be generated using the `run_geelite` function:

```r
# Building or updating the database:
run_geelite(path = "path/to/db")
```
[R code]

Re-running the `run_geelite` function updates the dataset. Both the building and updating processes can be easily handled through the corresponding CLI script, as shown below[12]:

```bash
# Building or updating the database:
Rscript cli/run_geelite.R
```
[Bash code]

(c) *Configuration Modification*

To update regions of interest, data sources, or the limit parameter, modify the configuration file using the `modify_config` function, then execute the `run_geelite` function to refresh the database. In the configuration, mean (`mean`) and minimum (`min`) statistics were initially selected for zonal statistics calculations. To tailor the database for the original task, the following code replaces the minimum statistic with the standard deviation (`sd`) in the configuration file and updates the database accordingly:

```r
# Modifying the configuration file:
modify_config(path = "path/to/db",
              keys = list(c("source", "MODIS/061/MOD13A2", "NDVI")),
              new_values = list(c("mean","sd")))

# Updating the database:
run_geelite(path = "path/to/db")
```
[R code]

Alternatively, the same modifications to the configuration file and database can be made via the CLI using the following code:

```bash
# Modifying the configuration file:
Rscript cli/modify_config.R --keys "list(c('source', 'MODIS/061/
     MOD13A2', 'NDVI'))" --new_values "list(c('mean', 'sd'))"

# Updating the database:
Rscript cli/run_geelite.R
```
[Bash code]

After the modifications, the generated SQLite database occupies 1.37 MB of disk space. Note that increasing the spatial resolution, number of indicators, aggregation methods, or geographic coverage will lead to greater disk usage. For example, raising the hexagonal resolution from size 3 to size 4 in the original configuration (reducing the bin size from approximately 12,393 to 1770 square kilometers) would increase the file size to 9.33 MB. Despite the higher resolution, the file size would still be manageable, even for multiple countries.

4.2. Reading and processing the database

The generated SQLite database (*data/geelite.db*) contains two tables: `grid`, which stores information about the H3 bins, and `MODIS/061/MOD13A2`, which holds the collected zonal statistics.[13] To read, aggregate, and pre-process the database in R, the `read_db` function

[11] The CLI scripts automatically load all required dependencies.

[12] Using the CLI script is particularly useful for scheduling regular database updates.

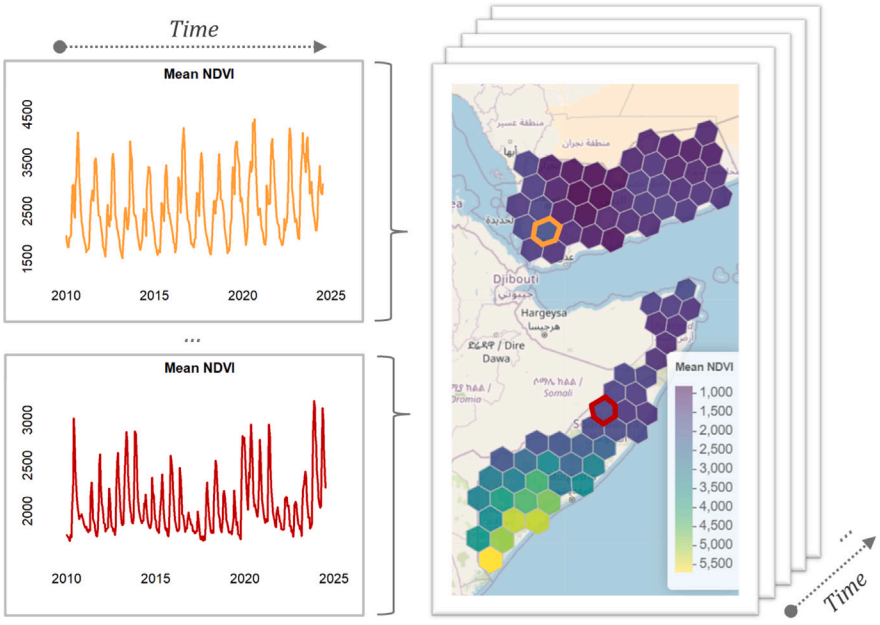

Fig. 2. Unscaled mean NDVI data retrieved from GEE (scale conversion factor: 0.00001).
Note: The generated database consists of forty-two H3 bins per country, each covering approximately 12,393 square kilometers. It tracks NDVI time series data across these bins, with 5306 consecutive values derived from linear interpolation of the original 335 measurements collected between January 1, 2010, and July 11, 2024.

from the `geeLite` package can be used. In this example, the dataset was converted to a daily frequency using the default pre-processing method, which applies linear interpolation:

```
# Reading SQLite tables:                                    [R code]
db <- read_db(path = "path/to/db", freq = "day")
```

This code snippet generates a list containing three elements: a simple features (sf) object representing the `grid` table, and two data frames corresponding to the variables `MODIS/061/MOD13A2/NDVI/mean` and `MODIS/061/MOD13A2/NDVI/sd`.

4.3. Visualizing the database

After joining these tables using the H3 index (`id`), the spatiotemporal characteristics of the generated database are illustrated in Fig. 2.[14]

As shown in Fig. 2, forty-two H3 bins were defined for both countries, each covering approximately 12,393 square kilometers. The number of time series generated for each bin corresponds to the number of variables collected. The original dataset, collected between January 1, 2010, and July 11, 2024, comprised 335 measurements. After converting the data to a daily frequency using linear interpolation, the number of measurements increased to 5306.

4.4. Automation

To automate the updating of the generated database, Linux crontab and the CLI version of the `run_geelite` function are used. For a monthly schedule, the following script can be utilized[15]:

```
# Monthly update with crontab:                              [Bash code]
(crontab -l 2>/dev/null; echo "0 0 1 * * Rscript cli/run_geelite.R") | crontab -
```

5. Recommended use, scalability, and limitations

The `geeLite` package is most effective when repeated extraction of zonal statistics from GEE must be automated and stored for long-term analysis in R. It targets workflows based on fixed spatial units — such as national or subnational administrative regions (admin0/admin1) represented on an H3 grid — where indicators are updated on a recurring basis and must remain reproducible and lightweight without heavy server infrastructure. A representative intended setting is recurring national monitoring in capacity-constrained environments, such as drought

[13] The contents of the SQLite database can be reviewed using the `fetch_vars` function.
[14] A code example for visualizing the mean NDVI values from the generated database is provided in Appendix B. To highlight the capabilities of the `geeLite` package, Fig. C.1 in Appendix C presents examples of higher-resolution H3 grid systems applied to NDVI data from Yemen.
[15] For more information about Linux crontab, see Bradley (2016).

monitoring carried out by the Somali National Bureau of Statistics as part of the Joint Monitoring Report (SNBS, 2024), where Earth observation indicators are refreshed regularly, early warning indicators need to be maintained, and their calculations need to be transparent.

The decision to use `geeLite` depends on the balance between analytical scale and workflow complexity. For one-off or exploratory work — e.g., testing alternative settings or doing ad-hoc processing — working directly with `rgee` is typically the most straightforward option. Even for small or medium-sized projects, however, `geeLite` becomes useful once the goal shifts from one-off analysis to data stewardship: maintaining an H3-based, versioned dataset that can be updated, extended, and reused over time, with changes tracked in a consistent way. Very large or high-frequency workflows — covering whole continents, fine H3 resolutions, or dense daily products — eventually exceed what a single SQLite file can handle efficiently; in such cases, server-grade solutions like PostGIS (PostGIS Development Group, 2025) or cloud databases are preferable. `geeLite` thus occupies the middle ground: structured enough for transparent, reproducible, and update-friendly workflows, while remaining simple to deploy locally.

Scalability mainly depends on spatial resolution (`resol`), batch size (`limit`), and image scale (`scale`). For smaller tasks — for instance a few countries, moderate H3 resolutions, and monthly or quarterly updates — `mode="local"` is usually the better default, since it avoids Drive transfer overhead and often runs faster end-to-end. As the database size grows, however, `mode="local"` can become bottlenecked by many small, chunk-level Earth Engine calls and the associated local merge/write work in a growing SQLite file.

In `mode="drive"`, `geeLite` still produces the same local SQLite database, but it issues fewer, larger Earth Engine exports via Google Drive and downloads only aggregated results; this can be substantially faster for large or high-frequency workloads because it reduces per-chunk overhead and can better exploit Earth Engine's parallel backend. In our benchmark, we replicated the illustrative configuration (Somalia and Yemen; `MODIS/061/MOD13A2` NDVI zonal `mean` and `sd`; H3 `limit` = 10 000), modifying only `start` to 2024-01-01 (run completed on 2025-12-05). `Local` mode was faster for small jobs (`resol` = 3: 10.6 s vs. 33.7 s), while `drive` became faster as task size increased (`resol` = 6: 31 min vs. 17.7 min), with identical outputs.

Reliable operation benefits from lean configurations, applying changes through the package interface (rather than manual edits), and treating interpolation or robust aggregations as deliberate analytical choices. Occasional SQLite maintenance — such as running `VACUUM` to compact/rebuild the database file — can help keep performance stable over time.

Because `geeLite` aggregates data into zonal summaries before export, pixel-level variation within zones is not retained. The single-file SQLite design is portable but unsuitable for concurrent multi-user access or multi-terabyte archives. Performance depends on local disk and memory, and upstream GEE catalog changes may require regenerating outputs. Within these bounds, `geeLite` offers a lightweight, reproducible way to build and maintain GEE-derived indicator databases directly in R.

6. Impacts and conclusions

This paper introduced `geeLite`, an R package that provides a configuration-driven pipeline for repeatable extraction of zonal statistics from Google Earth Engine (GEE) and stores results in a lightweight, incrementally updated SQLite database. The workflow targets analyses on stable H3 grid cells generated within user-selected ISO-defined regions (e.g., countries or first-level subdivisions), with built-in support for incremental updates, reproducible time-series construction, and tracked data revisions. Key practical benefits include:

- **Streamlined extraction from GEE:** `geeLite` reduces reliance on ad-hoc scripts by providing a structured pipeline for defining variables and regions once, then rebuilding or updating outputs consistently over time.
- **Support for regularly updated longitudinal indicators:** The package is well-suited to workflows where indicators are refreshed on a schedule (e.g., monthly or quarterly products), while keeping earlier versions traceable and comparable.
- **Portable and reproducible storage:** SQLite-backed outputs are self-contained and easy to move between machines or share within a team, facilitating replication of analyses without requiring server-side database infrastructure.
- **A middle ground between one-off scripting and heavier systems:** For small-to-medium scale projects, `geeLite` offers more structure than direct `rgee` usage while remaining simpler to deploy than PostGIS or cloud database solutions.

In summary, `geeLite` enables persistent, update-friendly storage of GEE-derived indicators on a fixed H3 grid with minimal local infrastructure. It is best suited to single-user or small-team workflows with periodic refreshes over stable H3 grids. For continent-scale coverage, very fine H3 resolutions, or high-frequency products, server-grade databases are preferable. While the current implementation is in R, the same configuration-driven approach could be extended to Python or JavaScript GEE workflows. Future work may include optional alternative backends (e.g., PostGIS) and stricter configuration validation for large-scale use cases.

CRediT authorship contribution statement

Marcell T. Kurbucz: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Project administration, Methodology, Investigation, Data curation, Conceptualization. **Bo Pieter Johannes Andrée:** Writing – review & editing, Writing – original draft, Validation, Supervision, Software, Project administration, Methodology, Investigation, Funding acquisition, Conceptualization.

Software and Data availability

- **Name of software:** `geeLite`
- **Developers:** Marcell T. Kurbucz and Bo Pieter Johannes Andrée
- **Contact:** m.kurbucz@ucl.ac.uk
- **Date first available:** May 7, 2025
- **Software required:** R version 4.3.2 or later (dependencies listed in the `DESCRIPTION` file); a Conda environment with `earthengine-api` (0.1.370), `ee_extra`, and `numpy`
- **Programming language:** R
- **Operating systems:** Linux, macOS, Windows

- **Source code:** https://github.com/mtkurbucz/geeLite (CRAN mirror: https://CRAN.R-project.org/package=geeLite)
- **Documentation:** https://github.com/mtkurbucz/geeLite/blob/master/README.md
- **Test files and examples:** Included in the GitHub repository
- **Data used:** Publicly accessible via Google Earth Engine (MODIS NDVI): https://developers.google.com/earth-engine/datasets/catalog/MODIS_061_MOD13A2

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Marcell T. Kurbucz reports financial support was provided by World Bank. Bo Pieter Johannes Andree reports financial support was provided by World Bank. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Post-processing with external files

The `init_postp` function creates a *postp* folder in the database root, containing an editable R script (*functions.R*) for user-defined post-processing functions and a JSON file (*structure.json*) that maps variables to those functions. Setting `postp_funs = "external"` in `read_db` automatically sources *functions.R* (the filename must remain *functions.R*) and, when aggregation is enabled (`freq` not `NULL`), applies each referenced function separately to the aggregated time series (`value`) of its mapped variable, returning one output series per function in the order listed in *structure.json*.

To illustrate the expected structure, consider an example where `"MODIS/061/MOD13A2/NDVI/mean"` is binarized using upper and lower deciles while all other variables remain unchanged. First, define `FUN_1 <- function(x) as.numeric(x < quantile(x, 0.1, na.rm = TRUE))` and `FUN_2 <- function(x) as.numeric(x > quantile(x, 0.9, na.rm = TRUE))` in *functions.R*. Then specify their use for the mean NDVI variable in *structure.json* as follows[16]:

```
{                                                          [JSON file]
  "default": null,
  "MODIS/061/MOD13A2/NDVI/mean": [
    ["FUN_1", "FUN_2"]
  ]
}
```

Appendix B. Code example for data visualization

Similar to Fig. 2, the mean NDVI values from the database generated in Section 4 can be visualized using the following R script:

```
# Install and load required packages:                      [R code]
# install.packages("sf")
# install.packages("dplyr")
# install.packages("leaflet")
library(sf)
library(dplyr)
library(leaflet)

# Read database and merge grid with MODIS data
db <- read_db(path = "path/to/db")
sf <- merge(db$grid, db$`MODIS/061/MOD13A2/NDVI/mean`, by = "id")

# Select the date to visualize
ndvi <- sf$`2020-01-01`

# Create a color palette function based on the values
color_pal <- colorNumeric(palette = "viridis", domain = ndvi)

# Create the leaflet map
leaflet(data = sf) %>%
  addTiles() %>%                          # Add base tiles
  addPolygons(
    fillColor = color_pal(ndvi),          # Fill color
    color = "#BDBDC3",                    # Border color
    weight = 1,                           # Border weight
    opacity = 1,                          # Border opacity
    fillOpacity = 0.9                     # Fill opacity
  ) %>%
  addScaleBar(position = "bottomleft") %>%  # Add scale bar
  addLegend(
    pal = color_pal,                      # Color palette
    values = ndvi,                        # Data values to map
    title = "Mean NDVI",                  # Legend title
    position = "bottomright"              # Legend position
  )
```

[16] Unlike in R, the absence of post-processing is denoted by lowercase `null` in the JSON file.

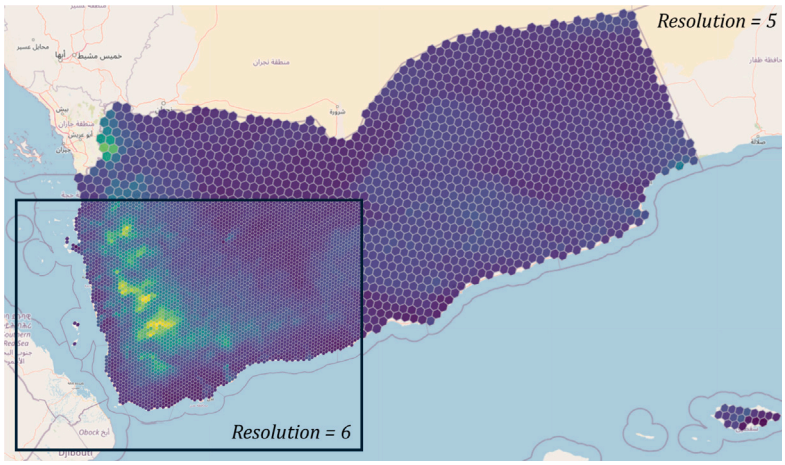

Fig. C.1. Examples of high-resolution H3 grid: unscaled mean NDVI data for Yemen retrieved from GEE (scale conversion factor: 0.00001).
Note: This figure demonstrates the capabilities of the `geeLite` package by collecting NDVI data for Yemen using high-resolution H3 grid systems. It illustrates two resolution levels: `resol = 5` and `resol = 6`, which correspond to grid cells with approximate areas of 252 square kilometers and 36 square kilometers, respectively.

Appendix C. High-resolution H3 grid

See Fig. C.1.

Data availability

The geeLite R package used in this study is open-source and freely available from CRAN and GitHub. The input data (MODIS NDVI) are publicly available via Google Earth Engine.

References

Amani, M., Ghorbanian, A., Ahmadi, S.A., Kakooei, M., Moghimi, A., Mirmazloumi, S.M., Moghaddam, S.H.A., Mahdavi, S., Ghahremanloo, M., Parsian, S., et al., 2020. Google earth engine cloud computing platform for remote sensing big data applications: A comprehensive review. IEEE J. Sel. Top. Appl. Earth Obs. Remote. Sens. 13, 5326–5350.
Andrée, B.P.J., Chamorro, A., Kraay, A., Spencer, P., Wang, D., 2020. Predicting food crises.
Andrée, B.P.J., Chamorro, A., Spencer, P., Koomen, E., Dogo, H., 2019. Revisiting the relation between economic growth and the environment: A global assessment of deforestation, pollution and carbon emission. Renew. Sustain. Energy Rev. 114, http://dx.doi.org/10.1016/j.rser.2019.06.028, URL: http://www.scopus.com/inward/record.url?eid=2-s2.0-85070497157&partnerID=MN8TOARS.
Apache Beam Community, 2025. Apache beam documentation. URL: https://beam.apache.org/documentation/. retrieved: December 1, 2025.
Arno, Z., Erickson, J., 2022. Tidyrgee: 'tidyverse' methods for 'earth engine'. R Package Version 0.1.0. https://github.com/r-tidy-remote-sensing/tidyrgee.
Aybar, C., Montero, D., Barja, A., Herrera, F., Gonzales, A., Espinoza, W., 2023. Combining r and earth engine. In: Cloud-Based Remote Sensing with Google Earth Engine: Fundamentals and Applications. Springer, pp. 629–651.
Aybar, C., Wu, Q., Bautista, L., Yali, R., Barja, A., 2020. Rgee: An R package for interacting with google earth engine. J. Open Source Softw. 5, 2272.
Banerjee, A., Ariz, D., Turyasingura, B., Pathak, S., Sajjad, W., Yadav, N., Kirsten, K.L., 2024. Long-term climate change and anthropogenic activities together with regional water resources and agricultural productivity in uganda using google earth engine. Phys. Chem. Earth Parts A/B/C 134, 103545.
Berner, L.T., Assmann, J.J., Normand, S., Goetz, S.J., 2023. LandsatTS': An r package to facilitate retrieval, cleaning, cross-calibration, and phenological modeling of landsat time series data. Ecography 2023, e06768.
Bradley, J., 2016. OS X Incident Response: Scripting and Analysis. Syngress.
Brovelli, M.A., Sun, Y., Yordanov, V., 2020. Monitoring forest change in the amazon using multi-temporal remote sensing data and machine learning classification on google earth engine. ISPRS Int. J. Geo Inf. 9, 580.
Burgueño, A.M., Aldana-Martín, J.F., Vázquez-Pendón, M., Barba-González, C., Jiménez Gómez, Y., García Millán, V., Navas-Delgado, I., 2023. Scalable approach for high-resolution land cover: A case study in the mediterranean basin. J. Big Data 10, 91.
Burke, M., Driscoll, A., Lobell, D.B., Ermon, S., 2021. Using satellite imagery to understand and promote sustainable development. Science 371, eabe8628.
Chen, S., Woodcock, C.E., Bullock, E.L., Arévalo, P., Torchinava, P., Peng, S., Olofsson, P., 2021. Monitoring temperate forest degradation on google earth engine using landsat time series analysis. Remote Sens. Environ. 265, 112648.
Fuller, M., Moser, M., Traverso, M., 2022. Trino: The Definitive Guide: SQL at Any Scale, on Any Storage, in Any Environment. O'Reilly Media, Inc.
GEE Community, 2025a. Arcgis-earthengine-toolbox: An arcgis pro toolbox for connection to google earth engine. URL: https://github.com/gee-community/arcgis-earthengine-toolbox. retrieved: December 1, 2025.
GEE Community, 2025b. Geetools: Google earth engine tools for the python API. URL: https://github.com/gee-community/geetools. retrieved: December 1, 2025.
GEE Community, 2025c. Google earth engine plugin for qgis. URL: https://github.com/gee-community/qgis-earthengine-plugin. retrieved: December 1, 2025.
Gorelick, N., Hancher, M., Dixon, M., Ilyushchenko, S., Thau, D., Moore, R., 2017. Google earth engine: Planetary-scale geospatial analysis for everyone. Remote Sens. Environ. 202, 18–27.
Hamidi, E., Peter, B.G., Muñoz, D.F., Moftakhari, H., Moradkhani, H., 2023. Fast flood extent monitoring with SAR change detection using google earth engine. IEEE Trans. Geosci. Remote Sens. 61, 1–19.
Hexagon Geospatial, 2024. ERDAS IMAGINE LiveLink for Google Earth Engine. Hexagon Geospatial, URL: https://bynder.hexagon.com/m/54525cd95849e827/original/Hexagon_SIG_Release_Guide_ERDAS_IMAGINE_2023_Update_2.pdf. Described in the ERDAS IMAGINE 2023 Update 2 Release Guide (v16.8), which documents the LiveLink integration and its Spatial Modeler operators. Retrieved: December 1, 2025.
Kazemi Garajeh, M., Haji, F., Tohidfar, M., Sadeqi, A., Ahmadi, R., Kariminejad, N., 2024. Spatiotemporal monitoring of climate change impacts on water resources using an integrated approach of remote sensing and google earth engine. Sci. Rep. 14, 5469.

Kong, D., 2022. rgee2. R Package Version 0.1.1, https://github.com/rpkgs/rgee2.
Kurbucz, M.T., Andrée, B.P.J., 2024. Geelite r package. URL: https://github.com/mtkurbucz/geeLite.
Marconcini, M., Metz-Marconcini, A., Esch, T., Gorelick, N., 2021. Understanding current trends in global urbanisation: The world settlement footprint suite. GI_Forum 9, 33–38.
Massicotte, P., South, A., 2024. Rnaturalearth: World map data from natural earth. URL: https://docs.ropensci.org/rnaturalearth/. R Package Version 1.0.1.9000, https://github.com/ropensci/rnaturalearth, https://docs.ropensci.org/rnaturalearthhires/.
Morales, N.S., Fernández, I.C., Durán, L.P., Pérez-Martínez, W.A., 2023. RePlant alfa: Integrating google earth engine and r coding to support the identification of priority areas for ecological restoration. Land 12, 303.
Müller, K., Wickham, H., James, D.A., Falcon, S., 2024. RSQLite: Sqlite interface for R. URL: https://rsqlite.r-dbi.org. R Package Version 2.3.7, https://github.com/r-dbi/RSQLite.
O'Brien, L., 2023. H3jsr: Access uber's H3 library. URL: https://obrl-soil.github.io/h3jsr/. R Package Version 1.3.1.
Penson, S., Lomme, M., Carmichael, Z., Manni, A., Shrestha, S., Andrée, B.P.J., 2024. A Data-Driven Approach for Early Detection of Food Insecurity in Yemen's Humanitarian Crisis. Technical Report, The World Bank.
PostGIS Development Group, 2025. Postgis 3.6 manual. URL: https://postgis.net/stuff/postgis-3.6-en.pdf. version 3.6.2dev.
Rocklin, M., et al., 2015. Dask: Parallel computation with blocked algorithms and task scheduling. In: SciPy. pp. 126–132.
SNBS, 2024. Somalia joint monitoring report: Update on food and nutrition security crisis risks, july 2024 – report #1. URL: https://nbs.gov.so/somalia-joint-monitoring-report/. SNBS = Somalia National Bureau of Statistics. Produced with contributions from the World Bank, FAO-FSNAU, SWALIM, and WFP. (Accessed 09 December 2025).
SSA Statistical Team, 2022. Saeplus. R Package Version 0.1.0. https://github.com/SSA-Statistical-Team-Projects/SAEplus.
Tamiminia, H., Salehi, B., Mahdianpari, M., Quackenbush, L., Adeli, S., Brisco, B., 2020. Google earth engine for geo-big data applications: A meta-analysis and systematic review. ISPRS J. Photogramm. Remote Sens. 164, 152–170.
Tavakkoli Piralilou, S., Einali, G., Ghorbanzadeh, O., Nachappa, T.G., Gholamnia, K., Blaschke, T., Ghamisi, P., 2022. A google earth engine approach for wildfire susceptibility prediction fusion with remote sensing data of different spatial resolutions. Remote. Sens. 14, 672.
Uber, T., 2021. H3: A hexagonal hierarchical geospatial indexing system.
Ushey, K., Allaire, J., Tang, Y., 2024. Reticulate: Interface to 'python'. URL: https://rstudio.github.io/reticulate/. R Package Version 1.35.0, https://github.com/rstudio/reticulate.
Velastegui-Montoya, A., Montalván-Burbano, N., Carrión-Mero, P., Rivera-Torres, H., Sadeck, L., Adami, M., 2023. Google earth engine: A global analysis and future trends. Remote. Sens. 15, 3675.
Vijayakumar, S., Saravanakumar, R., Arulanandam, M., Ilakkiya, S., 2024. Google earth engine: empowering developing countries with large-scale geospatial data analysis—a comprehensive review. Arab. J. Geosci. 17, 139.
Wang, D., Andrée, B.P.J., Chamorro, A.F., Spencer, P.G., 2022. Transitions into and out of food insecurity: A probabilistic approach with panel data evidence from 15 countries. World Dev. 159, 106035. http://dx.doi.org/10.1016/j.worlddev.2022.106035, URL: https://www.sciencedirect.com/science/article/pii/S0305750X2200225X.
Wang, L., Diao, C., Xian, G., Yin, D., Lu, Y., Zou, S., Erickson, T.A., 2020. A summary of the special issue on remote sensing of land change science with google earth engine.
Wickham, H., 2011. Testthat: Get started with testing. R J. 3, 5.
Wickham, H., Hester, J., Chang, W., Hester, M.J., 2022. Package 'devtools'.
Wu, Q., 2020. Geemap: A python package for interactive mapping with google earth engine. J. Open Source Softw. 5, 2305.
Wu, Q., Lane, C.R., Li, X., Zhao, K., Zhou, Y., Clinton, N., DeVries, B., Golden, H.E., Lang, M.W., 2019. Integrating lidar data and multi-temporal aerial imagery to map wetland inundation dynamics using google earth engine. Remote Sens. Environ. 228, 1–13.
Zaharia, M., Xin, R.S., Wendell, P., Das, T., Armbrust, M., Dave, A., Meng, X., Rosen, J., Venkataraman, S., Franklin, M.J., et al., 2016. Apache spark: A unified engine for big data processing. Commun. ACM 59, 56–65.
Zheng, Z., Wu, Z., Chen, Y., Yang, Z., Marinello, F., et al., 2021. Analyzing the ecological environment and urbanization characteristics of the yangtze River Delta urban agglomeration based on google earth engine. Acta Ecol. Sin. 41, 717–729.